P2363R1
Extending associative containers with the remaining heterogeneous overloads

Published Proposal, 2021-09-15

This version:
http://wg21.link/P2363R1

Authors:
Konstantin Boyarinov (Intel)
Sergey Vinogradov (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The authors propose heterogeneous overloads for the remaining member functions in ordered and unordered associative containers.

Table of Contents

§ 1. Motivation

[N3657] and [P0919R0] introduced heterogeneous lookup support for ordered and unordered associative containers in C++ Standard Library. [P2077R2] proposed heterogeneous erasure overloads. As a result, a temporary key object is not created when a different (but comparable) type is provided as a key to the member function. But there are still no heterogeneous overloads for methods such as `insert_or_assign`, `operator[]` and others.

Consider the following example:

```
void foo(std::map<std::string, int, std::less<void>>& m) {
    const char* lookup_str = "Lookup_str";
    auto it = m.find(lookup_str); // matches to template overload
    // ...
    auto result = m.insert_or_assign(lookup_str, 1); // no heterogeneous alternative
}
```

Function `foo` accepts a reference to the `std::map<std::string, int>` object. A comparator for the map is `std::less<void>`, which provides `is_transparent` valid qualified-id, so the `std::map` allows using heterogeneous overloads with `Key` template parameter for lookup functions, such as `find`, `upped_bound`, `equal_range`, etc.

In the example above, the `m.find(lookup_str)` call does not create a temporary object of the `std::string` to perform a lookup. But the `m.insert_or_assign(lookup_str, 1)` call causes implicit conversion from `const char*` to `std::string` even if the insertion will not take place. The allocation and construction (as well as subsequent destruction and deallocation) of the temporary object of `std::string` *or any custom object* might be quite expensive and reduce the program performance.

All the proposed APIs in this paper allow to avoid mentioned performance penalty.

§ 2. Prior work

Heterogeneous lookup overloads for ordered and unordered associative containers were added into C++ standard. For example:

```
template <typename K>
iterator find(const K& x);
```

The corresponding overloads were added for `count`, `contains`, `equal_range`, `lower_bound` and `upper_bound` member functions.

[P2077R2] proposed heterogeneous erasure overloads for ordered and unordered associative containers:

```
template <typename K>
size_type erase(K&& x);

template <typename K>
node_type extract(K&& x);
```

Constraints:

- qualified-id `Compare::is_transparent` is valid and denotes a type for ordered associative containers
- qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types for unordered associative containers

where `Compare` is a type of comparator passed to an ordered container, `Hash` and `Pred` are types of hash function and key equality predicate passed to an unordered container respectively.

§ 3. Proposal overview

We propose to add heterogeneous overloads for the following member functions:

- `insert` in `std::set` and `std::unordered_set`
- `insert_or_assign`, `try_emplace`, `operator[]`, `at` in `std::map` and `std::unordered_map`
- `bucket` in `std::unordered_set`, `std::unordered_map`, `std::unordered_multiset` and `std::unordered_multimap`

§ 3.1. `try_emplace`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename... Args>
std::pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template <typename K, typename... Args>
iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

Effects: If the map already contains an element whose key compares equivalent with `k`, there is no effect. Otherwise, inserts an object of type `value_type` constructed with `std::piecewise_construct, std::forward_as_tuple(std::forward<K>(k)), std::forward_as_tuple(std::forward<Args>(args)...)`.

Constraints:

1. For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
   For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types
2. `std::is_convertible_v<K&&, const_iterator>` and `std::is_convertible_v<K&&, iterator>` are both `false`.

Constraint 2 is introduced to maintain backward compatibility and makes homogeneous overload to be called when `K&&` is convertible to `const_iterator` or to `iterator`.

§ 3.2. `insert_or_assign`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename M>
std::pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template <typename K, typename M>
iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

Effects: If the map already contains an element `e` whose key compares equivalent with `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

Backward compatibility problem in case of `K&&` convertibility to `iterator` or `const_iterator` will not take place because the number of arguments is fixed - call with three arguments would always consider the overloads with `hint` only.

§ 3.3. `operator[]`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& operator[](K&& k);
```

Effects: Equivalent to: `return try_emplace(std::forward<K>(k)).first->second`.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.4. `at`

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& at(const K& k);

template <typename K>
const mapped_type& at(const K& k) const;
```

Returns: A reference to the `mapped_type` corresponding to `k` in `*this`.

Throws: An exception object of type `std::out_of_range` if no such element is present.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.5. `bucket`

We propose the following API (`std::unordered_map`, `std::unordered_multimap`, `std::unordered_set` and `std::unordered_multiset`):

```
template <typename K>
size_type bucket(const K& k) const;
```

Returns: The index of the bucket in which elements with keys compared equivalent with `k` would be found, if any such element existed.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are

valid and denote types

§ 3.6. `insert`

We considered to add heterogeneous overloads of `insert` for all associative containers, but found the applicability only for `std::set` and `std::unordered_set` with the following API:

```
template <typename K>
std::pair<iterator, bool> insert(K&& k);

template <typename K>
iterator insert(const_iterator hint, K&& k);
```

Effects: If the set already contains an element which compares equivalent with `k`, there is no effect. Otherwise, inserts an object constructed with `std::forward<K>(k)` into the set.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type
  For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

Adding heterogeneous `insert` overload makes no sence for associative containers with non-unique keys (`std::multimap`, `std::multiset`, `std::unordered_multimap` and `std::unordered_multiset`) because the insertion will be successful in any case and the key would be always constracted. All additional overheads introduced by `insert` can be mitigated by using `emplace`.

For `std::map` and `std::unordered_map`, heterogeneous insertion also makes no sence since heterogeneous `try_emplace` covers the relevant use cases.

§ 3.7. Design decisions

We do not introduce the following constraint: `std::is_constructible_v<value_type, K&&, some-arguments> == true` for `try_emplace`, `insert_or_assign`, `operator[]` and `insert` member functions because the only case when the homogeneous overload of the corresponding member function should be selected is when `K&&` is convertible to `key_type`, but `key_type` is not constructible from `K&&`. We do not consider such a scenario important to maintain.

The expression `std::is_constructible_v<value_type, K&&, some-arguments> == true` is considered as a precondition for heterogeneous overload.

§ 3.8. Proposed API summary

The proposed heterogeneous overloads for various methos are summarized in the table below:

Member function	std::set	std::multiset	std::map	std::multimap	std::unordered_set	std::unordered_multiset	std::unor
try_emplace	Not available	Not available	K&&	Not available	Not available	Not available	K&&
insert_or_assign	Not available	Not available	K&&	Not available	Not available	Not available	K&&
operator[]	Not available	Not available	K&&	Not available	Not available	Not available	K&&
at	Not available	Not available	const K&	Not available	Not available	Not available	const K&
bucket	Not available	Not available	Not available	Not available	const K&	const K&	const K&
insert	K&&	Not applicable	Not applicable	Not applicable	K&&	Not applicable	Not applic

§ 4. Formal wording

Below, substitute the character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 4.1. Modify class template `map` synopsis [**map.overview**]

[...]

```
// [map.access], element access
mapped_type& operator[](const key_type& x);
mapped_type& operator[](key_type&& x);

template<class K> mapped_type& operator[](K&& x);

mapped_type&         at(const key_type& x);
const mapped_type&   at(const key_type& x) const;

template<class K> mapped_type&       at(const K& x);
template<class K> const mapped_type& at(const K& x) const;
```

[...]

```
template<class... Args>
  pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
  pair<iterator, bool> try_emplace(key_type&& k, Args&&.. args);

template<class K, class... Args>
  pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template<class... Args>
  iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
  iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);

template<class K, class... Args>
  iterator try_emplace(const_iterator hint, K&& k, Args&&... args);

template<class M>
  pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
  pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);

template<class K, class M>
  pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template<class M>
  iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
  iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

template<class K, class M>
  iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

§ 4.2. Modify [**map.access**]

> [...]
>
> ```
> mapped_type& operator[](key_type&& x);
> ```
>
> — *Effects*: Equivalent to: `return try_emplace(move(x)).first->second;`
>
> ```
> template<class K> mapped_type& operator[](K&& x);
> ```
>
> — *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
> — *Effects*: Equivalent to: `return try_emplace(forward<K>(x)).first->second;`
>
> ```
> mapped_type&       at(const key_type& x);
> const mapped_type& at(const key_type& x) const;
> ```
>
> [...]
> — *Complexity*: Logarithmic.
>
> ```
> template<class K> mapped_type&       at(const K& x);
> template<class K> const mapped_type& at(const K& x) const;
> ```
>
> — *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
> — *Returns*: A reference to the `mapped_type` corresponding to `x` in `*this`.
> — *Throws*: An exception object of type `out_of_range` if no such element is present.
> — *Complexity*: Logarithmic.

§ 4.3. Modify [**map.modifiers**]

```
  template<class... Args>
    pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
  template<class... Args>
    iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class... Args>
    pair<iterator, bool> try_emplace(K&& k, Args&&... args);
  template<class K, class... Args>
    iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

— *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both `false`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `map` from `piecewise_construct, forward_as_tuple(forward<K>(k)), forward_as_tuple(forward<Args>(args)...)`.
— *Effects*: If the map already contains an element whose key is equivalent to `k`, there
Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_t`
`forward_as_tuple(std::forward<Args>(args)...)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

[...]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class M>
    pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
  template<class K, class M>
    iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

— *Constraints*: *Qualified-id* `Compare::is_transparent` is valid and denotes a type.
— *Mandates*: `is_assignable_v<mapped_type&, M&&>` is `true`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `map` from `forward<K>(k), forward<M>(obj)`.
— *Effects*: If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

§ 4.4. Modify class template `set` synopsis [**set.overview**]

> [...]
>
> ```
> pair<iterator, bool> insert(const value_type& x);
> pair<iterator, bool> insert(value_type&& x);
>
> template<class K> pair<iterator, bool> insert(K&& x);
>
> iterator insert(const_iterator hint, const value_type& x);
> iterator insert(const_iterator hint, value_type&& x);
>
> template<class K> iterator insert(const_iterator hint, K&& x);
> ```

§ 4.5. Add paragraph **Modifiers** into [**set.overview**]

> **`  Erasure  [set.erasure]`**
>
> [...]
>
> **`  Modifiers  [set.modifiers]`**
>
> ```
> template<class K> pair<iterator, bool> insert(K&& x);
> template<class K> iterator insert(const_iterator hint, K&& x);
> ```
>
> `  Constraints: Qualified-id Compare::is_transparent is valid and denotes a type.`
> `  Preconditions: value_type is Cpp17EmplaceConstructible into set from x.`
> `  Effects: Inserts a value_type object t constructed with std::forward<K>(x) if and only if there is no element in the container that is equivalent to`
> `  Returns: In the first overload, the bool component of the returned pair is true if and only if the insertion took place. The returned iterator points to the set element that is equivalent to x.`
> `  Complexity: Logarithmic.`

§ 4.6. Modify [**tab.container.hash.req**]

Table **[illegible]**: **Unordered associative container requirements (in addition to container) [tab:container.hash.req]**

Expression	Return type	Assertion/ note pre- / post-condition	Complexity
[...]			
`b.bucket(k)`	`size_type`	*Preconditions*: `b.bucket_count() > 0`. *Returns*: The index of the bucket in which elements with keys equivalent to `k` would be found, if any such element existed. *Postconditions*: The return value shall be in the range `[0, b.bucket_count())`.	Constant
`a_tran.bucket(ke)`	`size_type`	*Preconditions*: `a_tran.bucket_count() > 0`. *Returns*: The index of the bucket in which elements with keys equivalent to `ke` would be found, if any such element existed. *Postconditions*: The return value shall be in the range `[0, a_tran.bucket_count())`.	Constant
[...]			

§ 4.7. Modify paragraph [illegible] in [**unord.req.general**]

[...]
The member function templates `find`, `count`, `equal_range`, ~~and~~ `contains` and `bucket` shall not participate in overload resolution unless the *qualified-ids* `Pred::is_transparent` and `Hash::is_transparent` are both valid and denote types ([illegible]).

§ 4.8. Modify class template `unordered_map` synopsis [**unord.map.overview**]

[...]

```
template<class... Args>
  pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
  pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);

template<class K, class... Args>
  pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template<class... Args>
  iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
  iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);

template<class K, class... Args>
  iterator try_emplace(const_iterator hint, K&& k, Args&&... args);

template<class M>
  pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
  pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);

template<class K, class M>
  pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template<class M>
  iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
  iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

template<class K, class M>
  iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

[...]

```
// [unord.map.elem], element access
mapped_type& operator[](const key_type& k);
mapped_type& operator[](key_type&& k);

template<class K> mapped_type& operator[](K&& k);

mapped_type& at(const key_type& k);
const mapped_type& at(const key_type& k) const;

template<class K> mapped_type& at(const K& k);
template<class K> const mapped_type& at(const K& k) const;
```

[...]

```
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;

template<class K> size_type bucket(const K& k) const;
```

§ 4.9. Modify [**unord.map.access**]

[...]

```
  mapped_type& operator[](key_type&& k);
```

- *Effects*: Equivalent to: `return try_emplace(move(k)).first->second;`

```
  template<class K> mapped_type& operator[](K&& k);
```

- *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
- *Effects*: Equivalent to: `return try_emplace(forward<K>(k)).first->second;`

```
  mapped_type&       at(const key_type& k);
  const mapped_type& at(const key_type& k) const;
```

[...]

- *Throws*: An exception object of type `out_of_range` if no such element is present.

```
  template<class K> mapped_type&       at(const K& k);
  template<class K> const mapped_type& at(const K& k) const;
```

- *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
- *Returns*: A reference to `x.second`, where `x` is the (unique) element whose key is equivalent to `k`.
- *Throws*: An exception object of type `out_of_range` if no such element is present.

§ 4.10. Modify [**unord.map.modifiers**]

```
  template<class... Args>
    pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
  template<class... Args>
    iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class... Args>
    pair<iterator, bool> try_emplace(K&& k, Args&&... args);
  template<class K, class... Args>
    iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

— *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both `false`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `piecewise_construct, forward_as_tuple(forward<K>(k)), forward_as_tuple(forward<Args>(args)...)`.
— *Effects*: If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

[...]

```
  template<class M>
    pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
  template<class M>
    iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

[...]

— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

```
  template<class K, class M>
    pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
  template<class K, class M>
    iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

— *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
— *Mandates*: `is_assignable_v<mapped_type&, M&&>` is `true`.
— *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `forward<K>(k), forward<M>(obj)`.
— *Effects*: If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `std::forward<K>(k), std::forward<M>(obj)`.
— *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
— *Complexity*: The same as `emplace` and `emplace_hint`, respectively.

§ 4.11. Modify class template `unordered_multimap` synopsis [**unord.multimap.overview**]

> [...]
>
> ```
> size_type bucket_size(size_type n) const;
> size_type bucket(const key_type& k) const;
>
> template<class K> size_type bucket(const K& k) const;
> ```

§ 4.12. Modify class template `unordered_set` synopsis [**unord.set.overview**]

> [...]
>
> ```
> pair<iterator, bool> insert(const value_type& obj);
> pair<iterator, bool> insert(value_type&& obj);
>
> template<class K> pair<iterator, bool> insert(K&& obj);
>
> iterator insert(const_iterator hint, const value_type& obj);
> iterator insert(const_iterator hint, value_type&& obj);
>
> template<class K> iterator insert(const_iterator hint, K&& obj);
> ```
>
> [...]
>
> ```
> size_type bucket_size(size_type n) const;
> size_type bucket(const key_type& k) const;
>
> template<class K> size_type bucket(const K& k) const;
> ```

§ 4.13. Add paragraph **Modifiers** into [**unord.set.overview**]

> **Erasure [set.erasure]**
>
> [...]
>
> **Modifiers [set.modifiers]**
>
> ```
> template<class K> pair<iterator, bool> insert(K&& obj);
> template<class K> iterator insert(const_iterator hint, K&& obj);
> ```
>
> *Constraints*: *Qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.
> *Preconditions*: `value_type` is *Cpp17EmplaceConstructible* into `unordered_set` from `x`.
> *Effects*: Inserts a `value_type` object `t` constructed with `std::forward<K>(x)` if and only if there is no element in the container that is equivalent to
> *Returns*: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the set element that is equivalent to `x`.
> *Complexity*: Average case - constant, worst case - linear.

§ 4.14. Modify class template `unordered_multiset` synopsis [**unord.multiset.overview**]

> [...]
>
> ```
> size_type bucket_size(size_type n) const;
> size_type bucket(const key_type& k) const;
>
> template<class K> size_type bucket(const K& k) const;
> ```

§ 5. Revision history

§ 5.1. R0 ➡ R1

- Removed `std::is_constructible_v` constraint. For more information see § 3.7 Design decisions.
- Added § 4 Formal wording.

§ References

§ Informative References

[N3657]
J. Wakely, S. Lavavej, J. Muñoz. Adding heterogeneous comparison lookup to associative containers (rev 4). 19 March 2013. URL: https://wg21.link/n3657

[P0919R0]
Mateusz Pusz. Heterogeneous lookup for unordered containers. 8 February 2018. URL: https://wg21.link/p0919r0

[P2077R2]
Konstantin Boyarinov, Sergey Vinogradov; Ruslan Arutyunyan. Heterogeneous erasure overloads for associative containers. 15 December 2020. URL: https://wg21.link/p2077r2